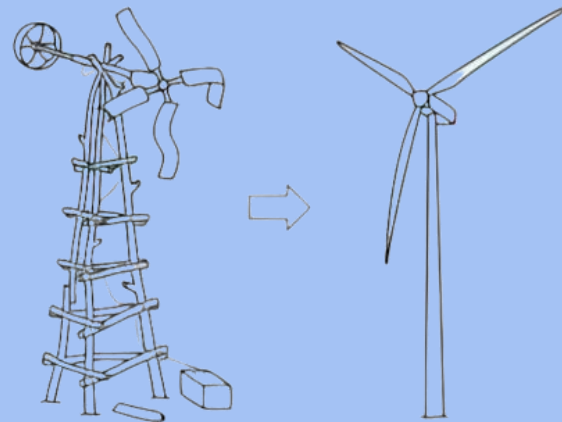


Code Smell & Refactoring



What is Code Smell?

A code smell is a design problem that makes code duplicate, complicated, bloated, or tightly coupled.

A code smell is any symptom in the source code of a program that may indicate a deeper problem. Code smells are usually not bugs because they are not technically incorrect and do not stop the program from working. Instead, they show weaknesses in the design that may slow down development or increase the risk of bugs and failures in the future

```
public abstract class AbstractCollection implements Collection {  
    public void addAll(AbstractCollection c) {  
        if (c instanceof Set) {  
            Set s = (Set)c;  
            for (int i=0; i < s.size(); i++) {  
                if (!contains(s.getElementAt(i))) {  
                    add(s.getElementAt(i));  
                }  
            }  
        }  
        else if (c instanceof List) {  
            List l = (List)c;  
            for (int i=0; i < l.size(); i++) {  
                if (!contains(l.get(i))) {  
                    add(l.get(i));  
                }  
            }  
        }  
        else if (c instanceof Map) {  
            Map m = (Map)c;  
            for (int i=0; i<m.size(); i++)  
                add(m.keys[i], m.values[i]);  
        }  
    }  
}
```

The diagram illustrates several code smells in the provided Java code snippet:

- Duplicated Code:** Indicated by a blue oval around the `if (c instanceof Set)` block and a purple oval around the `if (!contains(s.getElementAt(i)))` condition.
- Duplicated Code:** Indicated by a purple oval around the `if (!contains(l.get(i)))` condition.
- Alternative Classes with Different Interfaces:** Indicated by a green oval around the `if (!contains(l.get(i)))` condition.
- Switch Statement:** Indicated by a red oval around the `if (c instanceof Map)` block.
- Inappropriate Intimacy:** Indicated by a red oval around the `add(m.keys[i], m.values[i]);` line.
- Long Method:** Indicated by a black oval around the entire `addAll` method.

What is Refactoring?

" Refactoring is the process of changing a software system in a way that does not change the external behavior of the code but improves its internal structure." – Martin Fowler



Example

1. Duplicate Code

Same code is repeated in multiple places.

```
def calculate_student1():
```

```
    marks = 80
```

```
    result = marks * 0.5
```

```
    print(result)
```

```
def calculate_student2():
```

```
    marks = 90
```

```
    result = marks * 0.5
```

```
    print(result)
```

Problem: The calculation code is duplicated.

2. Complicated Code

Code is difficult to understand because of unnecessary complexity.

```
def check_number(num):
```

```
    if num > 0:
```

```
        if num % 2 == 0:
```

```
            if num < 100:
```

```
                print("Valid number")
```

Problem: Too many nested conditions make the code harder to read.

A short history of Code Smells

- If it stinks, change it!
- Kent Beck coined the term code smell to signify something in code that needed to be changed.



Common Code Smells

- Black sheep
- Comments
- Contrived complexity
- Data class
- Data clumps
- Dead code
- Duplicated code
- Divergent change
- Feature envy
- Inappropriate naming
- Large class
- Lazy class
- Long method
- Long parameter list
- Middle man
- Oddball solution
- Primitive obsession
- Refused bequest
- Shotgun Surgery
- Switch statements

Common Refactoring Techniques

- Add Parameter
- Collapse Hierarchy
- Encapsulate Collection
- Encapsulate Field
- Extract Class
- Extract Interface
- Extract Method
- Extract Subclass
- Extract Superclass
- Extract Variable
- Inline Class
- Inline Method
- Inline Temp
- Introduce Assertion
- Introduce Null Object
- Introduce Parameter Object
- Move Field
- Move Method
- Preserve Whole Object
- Pull Down Field
- Pull Down Method
- Pull Up Constructor Body
- Pull Up Field
- Pull Up Method
- Push Down Field
- Push Down Method
- Remove Control Flag
- Remove Middle Man
- Remove Parameter
- Rename Method
- Replace Conditional with Polymorphism
- Replace Constructor with Factory Method
- Replace Error Code with Exception
- Replace Exception with Test
- Replace Inheritance with Delegation
- Replace Method with Method Object
- Replace Subclass with Fields
- Replace Temp with Query
- Replace Type Code with Class
- Replace Type Code with State/Strategy
- Replace Type Code with Subclasses
- Substitute Algorithm

Inappropriate Naming

- Names given to variables (fields), methods or class should be clear and meaningful.
- A variable, field, class name should say exactly what it is.
- **Which is better?**
`private double s;` OR `private double salary;`
- A method should say exactly what it does.
- **Which is better?**
`public double calc(double s);` OR
`public double calculateFederalTaxes (double salary);`

Refactoring Techniques:

- ❑ Rename Variables, Fields, Method, Class

Practice: Refactoring Inappropriate Naming

Identify and refactor inappropriate names for variables, methods, and classes to enhance the clarity and readability of the code.



```
public class M
{
    public double M2(int x, int y)
    {
        return x - y
    }

    private int Calc(int a, int b)
    {
        return a + b;
    }
}
```

```
public class Calculator
{
    public double subtractNumbers(int firstNumber, int secondNumber)
    {
        return firstNumber - secondNumber;
    }

    private int addNumbers(int firstNumber, int secondNumber)
    {
        return firstNumber + secondNumber;
    }
}
```

Comments

- Comments are often used as deodorant.
- Comments can hide problems in the code instead of solving them. They may show that the code does not clearly explain the programmer's idea or purpose.
- Try to make your code **self-documenting** or **intention-revealing** so that the purpose of the code is clear without needing extra explanations.
- When you feel like writing a comment, first try to **refactor the code** to make it clearer and easier to understand.

Refactoring Techniques

- ☐ Extract Method
- ☐ Rename Method



Comments (Cont'd)

Example: Code with unnecessary comments (problem)

```
# Check if the user is allowed to vote  
if age >= 18:  
    print("Allowed")
```

Improved self-documenting code (intention-revealing)

```
def is_eligible_to_vote(age):  
    return age >= 18  
if is_eligible_to_vote(age):  
    print("Allowed")
```

Comments (Cont'd)

```
void List::add(string element)
{
    if (m_readOnly)
        return;
    if (shouldGrow())
        grow();
    storeElement(element);
}
```

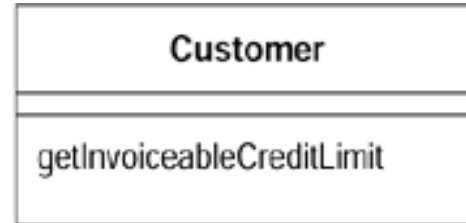
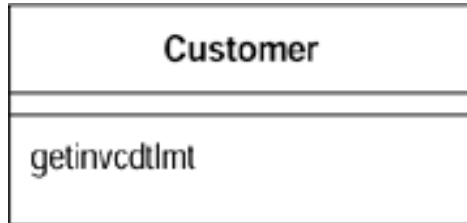
```
bool List::shouldGrow()
{
    return (m_size + 1) >
capacity();
}
```

```
void List::grow()
{
    m_capacity += 10;
    string *newElements = new string[m_capacity];
    for(int i = 0; i < m_size; i++)
        newElements[i] = m_elements[i];
    delete [] m_elements;
    m_elements = newElements;
}
```

```
void List::storeElement(string element)
{
    m_elements[m_size++] = element;
}
```

Refactoring Technique: Rename Method

Rename Method



Refactoring Technique: Extract Method

Extract Method is a refactoring technique that involves moving a section of code from an existing method into a new separate method with a meaningful name to improve code readability, maintainability, and reduce complexity.



```
def print_owning(amount):  
    print_banner()  
    # print_details  
    print("name:", name)  
    print("amount:", amount)
```

```
def print_owning(amount):  
    print_banner()  
    print_details(amount)  
  
def print_details(amount):  
    print("name:", name)  
    print("amount:", amount)
```

Practice: Refactoring Comments

Refactor the code by removing the comments and making the code self-explanatory through better naming and structure.

```
public class ShoppingCart
{
    private List<Item> items = new List<Item>();
    private int maxCapacity = 10;

    public void AddItem(Item item)
    {
        // Check if the cart has space
        if (items.Count >= maxCapacity)
        {
            // If full, expand the cart
            ExpandCart();
        }

        // Add item to the cart
        items.Add(item);
    }

    private void ExpandCart()
    {
        // Increase max capacity
        maxCapacity += 10;
        Console.WriteLine("Cart capacity expanded.");
    }
}
```

```
public class ShoppingCart
{
    private readonly List<Item> items = new();
    private const int CapacityIncrement = 10;

    public void AddItem(Item item)
    {
        EnsureCapacity();
        items.Add(item);
    }

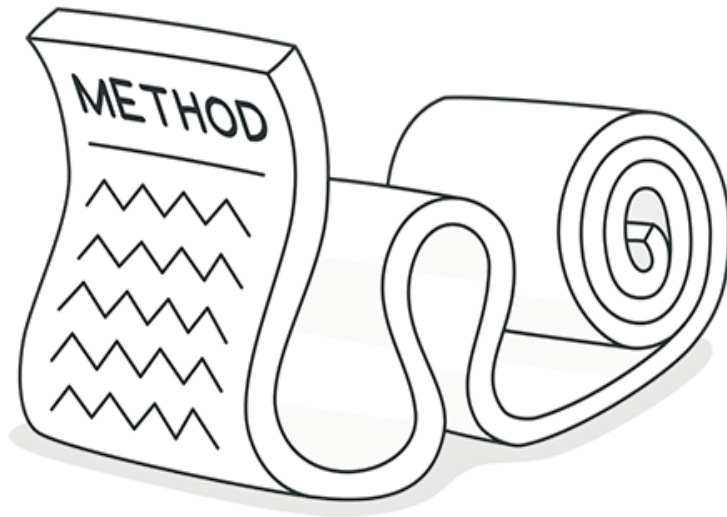
    private void EnsureCapacity()
    {
        if (items.Count >= maxCapacity)
            GrowCapacity();
    }

    private void GrowCapacity()
    {
        maxCapacity += CapacityIncrement;
        Console.WriteLine($"Cart capacity expanded to {maxCapacity}.");
    }
}
```

Long Method

A method is long when it is:

- Hard to read
- Hard to change
- Difficult to test
- Low reusability
- Trying to do too much
- Tendency for bugs
- Too hard to quickly comprehend



Long Method (Cont'd)

No. of lines

- 10–20 lines: Often considered an ideal
- A method with more than 30 lines is considered too long and likely contains more than one responsibility. It may be difficult to understand, maintain, and modify.
- A method with more than 50 lines is a strong code smell, indicating that it is handling too many responsibilities or has too much complexity.

Refactoring Technique:

Extract Method — Break a long method into smaller methods, where each method performs a single clear task.

Long Method (Cont'd)

No. of Parameters

- 3–4 parameters is nice
- more than 3–4 parameters is typically considered a code smell. is relying too heavily on external data

No. of Local Variables

- 5–7 local variables is fine
- More than 7 variable indicates that the method might be doing too much. It often leads to increased complexity, making the code harder to follow and maintain

3–4 parameters & 5–7 local variables are common thresholds for identifying a long method

Long Method (Cont'd)

The ideal goal is to keep the method's signature and logic concise, readable, and focused on a single responsibility

Cyclomatic Complexity - Long Method (Cont'd)

Cyclomatic Complexity is a software metric that measures the number of independent paths through a program's code and determines the minimum number of test cases required.

A high cyclomatic complexity (e.g., greater than 10) suggests the method is too complex and difficult to test.

Return Statements & Control Flow - Long Method (Cont'd)

Multiple return statements or early exits from a method can indicate a method trying to do too much. A clear and singular flow of control (with one return point) is often preferable

Nesting Levels (Indentation Depth) - Long Method (Cont'd)

Deep nesting occurs when code contains **many levels of nested conditions or loops** inside each other. It makes the code difficult to read, understand, test, and maintain

Deep nesting (e.g., more than 2-3 levels of if, for, while, or switch) can make the code harder to maintain.

Long Method (Cont'd)

To define a long method, consider:

- Line count (general length).
- Number of parameters (avoid more than 3-4).
- Number of local variables (avoid more than 5-7).
- Cyclomatic complexity (keep under 10).
- Nesting levels (avoid deep indentation).
- Control flow complexity (multiple returns, etc.).
- Single Responsibility Principle (SRP) (ensure the method has one responsibility).
- Duplication (refactor repeated logic).

Long Method (Cont'd)

This “smell” remains unnoticed until the method turns into an ugly, oversized beast.

Refactoring Techniques:

- Extract Method
- Replace Temp with Query
- Introduce Parameter Object
- Preserve Whole Object
- Replace Method with Method Object.
- Decompose Conditional

Refactoring Technique: Extract Method

```
private String toStringHelper(StringBuffer
result) {
    result.append("<");
    result.append(name);
    result.append(attributes.toString());
    result.append(">");
    if (!value.equals(""))
        result.append(value);
    Iterator it = children().iterator();
    while (it.hasNext()) {
        TagNode node = (TagNode)it.next();
        node.toStringHelper(result);
    }
    result.append("</");
    result.append(name);
    result.append(">");
    return result.toString();
}
```

Refactoring Technique: Extract Method (Cont'd)

```
def print_invoice(price, quantity):  
    print("Invoice")  
  
    total = price * quantity  
    print("Total:", total)  
  
    print("Thank you for shopping!")
```

```
def print_invoice(price, quantity):  
    print_total(price, quantity)  
    print_message()  
  
def print_total(price, quantity):  
    total = price * quantity  
    print("Total:", total)  
  
def print_message():  
    print("Thank you for shopping!")
```

Refactoring Technique: Replace Temp with Query

Replace Temp with Query is a refactoring technique where you remove a temporary variable that stores the result of a calculation and replace it with a separate method (query) that returns the value directly.

```
double basePrice = _quantity *  
_itemPrice;  
if(basePrice > 1000)  
{  
    return basePrice * 0.95;  
}  
else  
{  
    return basePrice * 0.98;  
}
```

```
if(getBasePrice() > 1000) {  
    return getBasePrice() * 0.95;  
}  
else {  
    return getBasePrice() * 0.98;  
}  
  
double getBasePrice() {  
    return _quantity * _itemPrice;  
}
```

Refactoring Technique: Introduce Parameter Object

BEFORE (Multiple Parameters)



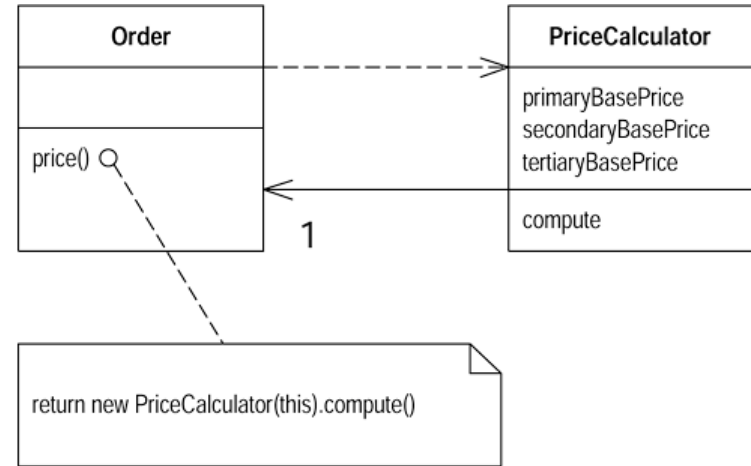
AFTER (Parameter Object)



Introduce Parameter Object is a refactoring technique where a group of related parameters passed to a method are replaced with a single object that contains those parameters.

Refactoring Technique: Replace Method with Method Object

```
class Order
{
    double Price() {
        double primaryBasePrice;
        double secondaryBasePrice;
        double tertiaryBasePrice;
        // long computation;
        ...
    }
}
```



Replace Method with Method Object is a refactoring technique where a complex method is moved into a separate class (method object). The new object stores the required data and performs the operation through its own method.

Refactoring Technique: Decompose Conditional

Decompose Conditional is a refactoring technique where a complicated if-then-else condition is divided into separate methods for the condition, then part, and else part.

```
if (date.before (SUMMER_START) || date.after(SUMMER_END))  
    charge = quantity * _winterRate + _winterServiceCharge;  
else  
    charge = quantity * _summerRate;  
-----  
-----  
if (notSummer(date))  
    charge = winterCharge(quantity);  
else  
    charge = summerCharge (quantity);
```

Practice: Refactor Long Method

You are given a method in a `PurchaseOrder` class that performs several operations, including calculations, condition checks, and updates. The method is long and difficult to follow. Your task is to refactor the code using appropriate techniques to make it cleaner and easier to maintain.

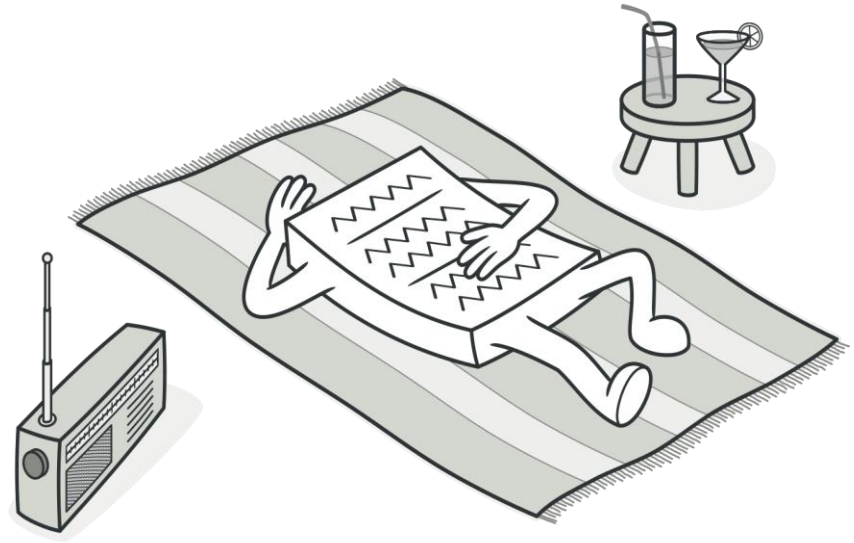
bit.ly/3YrrkvB

Code Smell: Lazy Class

A Lazy Class is a class that does not do enough work or has very little responsibility. It adds unnecessary complexity to the system and may be removed or combined with another class.

Refactoring Techniques:

- Inline Class
- Collapse Hierarchy



Code Smell: Lazy Class (Cont'd)

```
class DiscountCalculator:
    def calculate(self, price):
        return price * 0.9

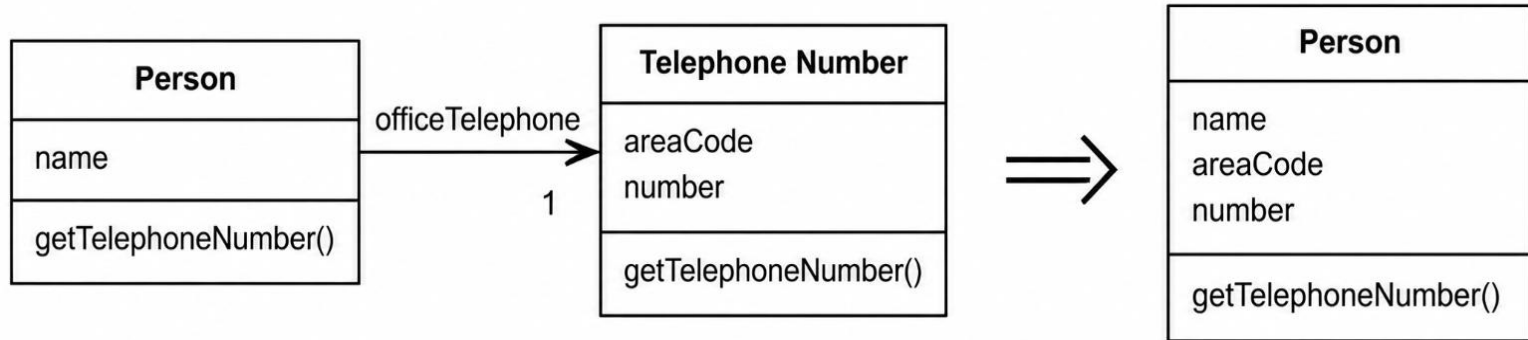
class ShoppingCart:
    def __init__(self, total):
        self.total = total

    def get_price(self):
        calculator = DiscountCalculator()
        return calculator.calculate(self.total)
```

```
class ShoppingCart:
    def __init__(self, total):
        self.total = total

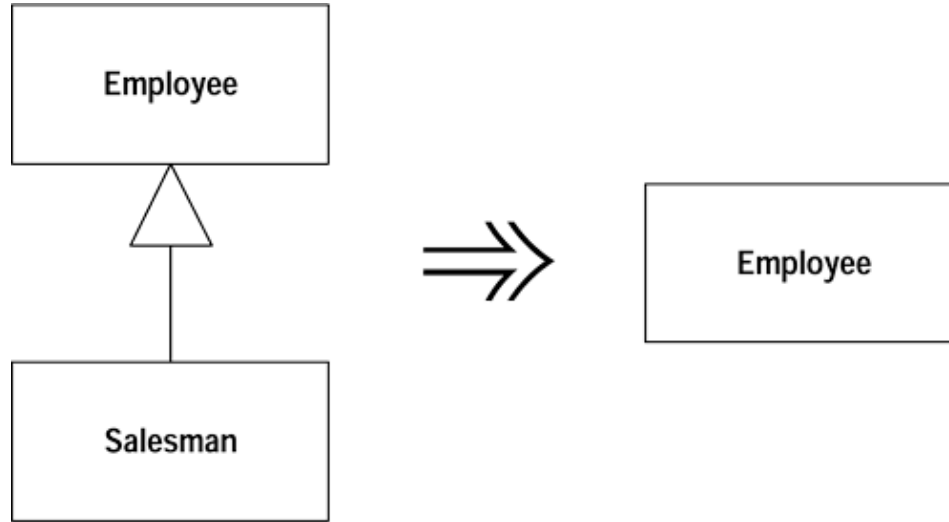
    def get_price(self):
        return self.total * 0.9
```

Refactoring Technique: Inline Class



Inline Class is a refactoring technique used when a class does very little work or has no meaningful responsibility. In this case, the class is removed, and its methods and fields are moved into another class.

Refactoring Technique: Collapse Hierarchy



Collapse Hierarchy is a refactoring technique used when a superclass and subclass are not different enough and the subclass does not provide any extra behavior. In this case, the two classes can be combined into a single class.

Code Smell: Middle man

An unnecessary layer that just passes calls along without adding any real value

Refactoring technique:
Remove middle man



Code Smell: Middle man (Cont'd)

```
class Employee:
    def get_salary(self):
        return self.salary

class EmployeeManager:
    def __init__(self, employee):
        self.employee = employee

    def get_salary(self):
        return self.employee.get_salary()
```

```
class Employee:
    def __init__(self, salary):
        self.salary = salary

    def get_salary(self):
        return self.salary
```

Code Smell: Middle man (Cont'd)

```
public class Client {  
    public void DisplayCustomer(int id) {  
        CusManager manager = new CusManager();  
        Console.WriteLine(manager.GetName(id));  
        Console.WriteLine(manager.GetAddress(id));  
    }  
}
```

Code Smell: Oddball Solution

- An oddball solution occurs particular problem is solved in multiple different ways throughout a codebase, rather than using a consistent approach.

Examples:

- Choosing different tools or libraries
- Implementing different coding styles or patterns for similar tasks.



Code Smell: Oddball Solution (Cont'd)

```
class Payment:
    def process(self, payment_type, amount):
        if payment_type == "credit_card":
            return amount * 0.95

        elif payment_type == "paypal":
            return amount * 0.90

        elif payment_type == "bitcoin": # Odd special case
            return amount - 10
```

```
class Payment:
    def __init__(self, discount):
        self.discount = discount

    def process(self, amount):
        return amount * (1 - self.discount)

credit_card = Payment(0.05)
paypal = Payment(0.10)

print(credit_card.process(100))
print(paypal.process(100))
```


Code Smell: Oddball Solution (Cont'd)

```
public class ContactForm {  
    public bool ValidateEmail(string email) {  
        //Using a simple string operation for email validation  
        return email.Contains("@") && email.Contains(".");  
    }  
}
```

Code Smell: Oddball Solution (Cont'd)

```
public class NewsletterSubscription {  
    public bool ValidateEmail(string email) {  
        // Using System.Net.Mail to validate email  
        try { var addr = new System.Net.Mail.MailAddress(email);  
            return addr.Address == email; }  
        catch {  
            return false;  
        }  
    }  
}
```

Practice 23: Refactor Oddball Solution

Refactoring technique:

- Extract class
- Extract Method to Utility

Practice:

bit.ly/3YuWand

Code Smell: Black Sheep



Code Smell: Black Sheep

A Black Sheep is a part of the code that is significantly different from others in the same group or structure. It does not follow the common design or behavior of the surrounding code.

Examples:

- A subclass that significantly differs from others in the hierarchy.
- A method in a class that is noticeably different from other methods in the same class.
- Lack of Single Responsibility Principle (SRP): A class or method handles multiple unrelated responsibilities instead of focusing on one specific purpose.

Refactoring Techniques -

- Move Method
- Extract Class

Code Smell: Black Sheep



```
public class FileManager
{
    public string FilePath { get; set; }

    public FileManager(string filePath)
    {
        FilePath = filePath;
    }

    public void ReadFile()
    {
        Console.WriteLine($"Reading the file: {FilePath}");
    }

    public void WriteFile(string content)
    {
        Console.WriteLine($"Writing to the file: {FilePath}");
    }

    public void SendEmail(string to, string subject, string body)
    {
        Console.WriteLine($"Sending an email to {to}: Subject - {subject}");
    }
}
```

Code Smell: Black Sheep



```
public class FileManager
{
    public string FilePath { get; set; }

    public FileManager(string filePath)
    {
        FilePath = filePath;
    }

    public void ReadFile()
    {
        Console.WriteLine($"Reading the file: {FilePath}");
    }

    public void WriteFile(string content)
    {
        Console.WriteLine($"Writing to the file: {FilePath}");
    }
}
```

Code Smell: Black Sheep



```
public class EmailSender
{
    public void SendEmail(string to, string subject, string body)
    {
        Console.WriteLine($"Sending an email to {to}: Subject - {subject}");
    }
}
```


Code Smell: Dead Code

Dead Code is code that is **no longer used in a system or related system**. It does not contribute to the functionality of the program and can be safely removed.

Problems caused by Dead Code:

- **Increased Complexity:**
Unused code makes the system larger and harder to understand.
- **Accidental Changes:**
Developers may modify or depend on unused code by mistake, causing unexpected problems.
- **More Dead Code:**
Unremoved unused code can encourage more unnecessary code to remain in the system.

Refactoring Technique:

Remove Dead Code — Delete unused methods, variables, classes, and code blocks to make the system cleaner and easier to maintain.

- Remedy



Code Smell: Dead Code (Cont'd)

```
def calculate_total(price, quantity):  
    total = price * quantity  
    return total  
  
print("Calculation completed") # Dead code
```

Code Smell: Feature Envy (Cont'd)

According to OOP principles, data and the logic that manipulates it should be encapsulated within the same class. So, a method that seems more interested in another class's data and behavior than its own indicates a design issue.

Feature Envy occurs when a method uses more data or methods from another class than from its own class.



Practice 24: Refactor Feature Envy (Cont'd)

Refactoring Techniques -

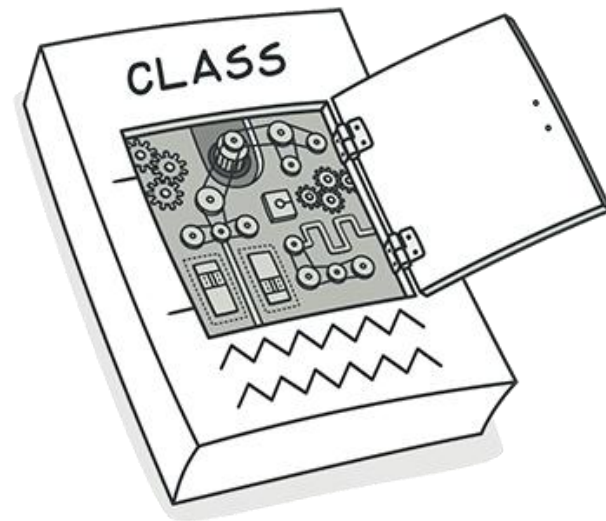
- Move Field
- Move Method
- Extract Class

Practice:

bit.ly/3YEIgPv

Code Smell: Divergent Change

Divergent Change is when many changes are made to a single class



Refactoring Techniques:

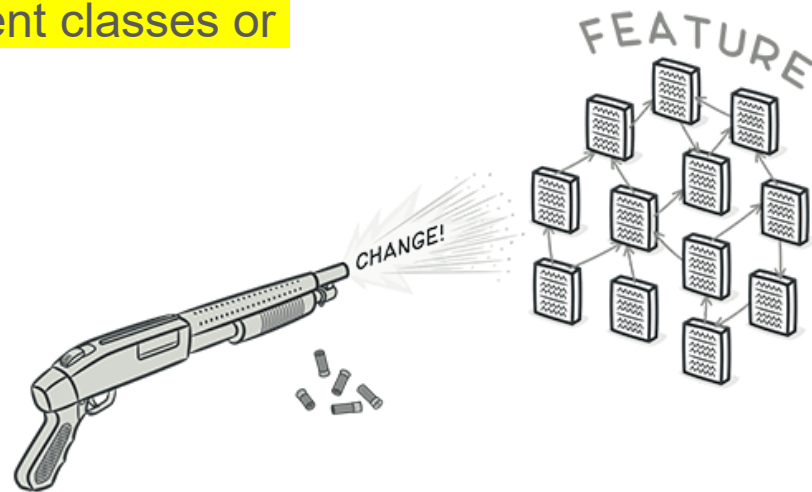
- Split up the behavior of the class via Extract Class.
- If different classes have the same behavior, you may want to combine the classes through inheritance (Extract Superclass and Extract Subclass).

Code Smell: Shotgun Surgery

Shotgun Surgery occurs when a small change in the system requires modifications in many different classes or files.

Refactoring Techniques:

- Use Move Method and Move Field to move existing class behaviors into a single class. If there's no class appropriate for this, create a new one.
- If moving code to the same class leaves the original classes almost empty, remove that unnecessary class by combining its remaining code with another class. This is called Inline Class.

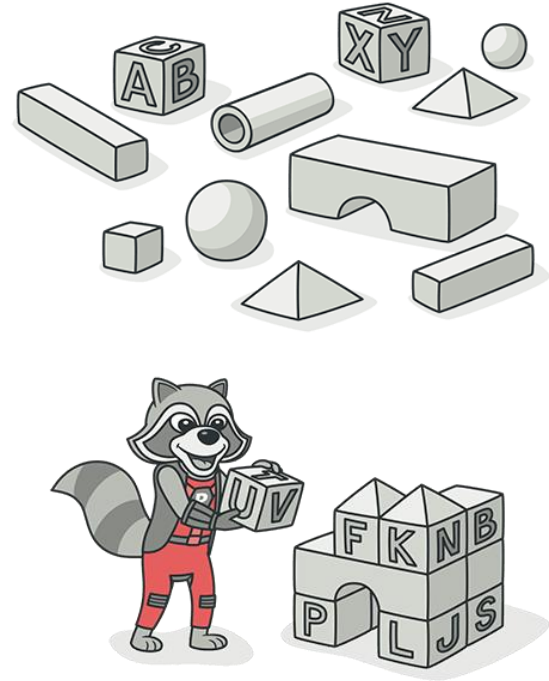


Code Smell: Primitive Obsession

Primitive Obsession occurs when a program uses too many basic data types (such as int, float, string, or boolean) instead of creating meaningful classes to represent real-world concepts.

Refactoring Techniques:

- Replace Data Value with Object
- Introduce Parameter Object or Preserve Whole Object.
- Replace Array with Object
- Think in OO Way



Code Smell: Speculative Generality

Speculative Generality is a code smell that occurs when developers add unnecessary generic code, abstractions, hooks, or special cases to support possible future requirements that are not currently needed. It increases complexity and makes the code harder to understand and maintain.

Refactoring Techniques:

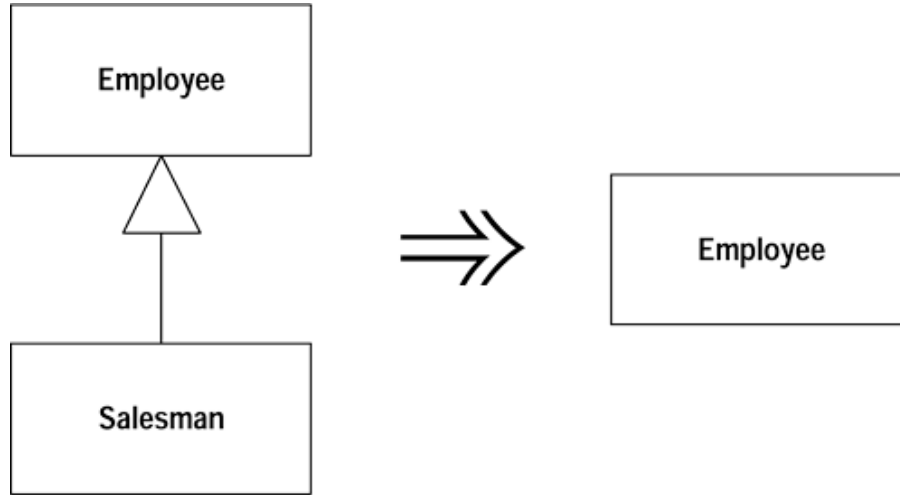
- Collapse Hierarchy
- Inline Class
- Remove Parameter
- Rename Method



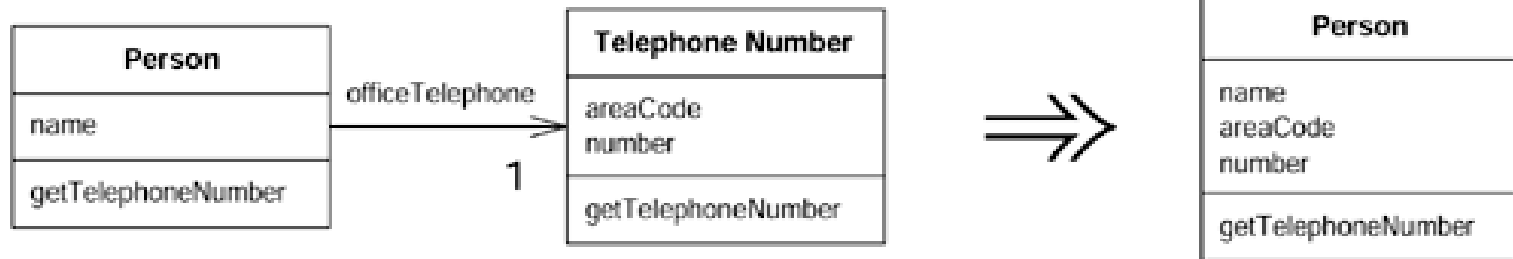
Code Smell: Speculative Generality (Cont'd)

```
public class Customer {
    private String name;
    private String address;
    private String salutation;
    private String otherDetails;
    private MailingSystem mailingSystem;
    public Customer(String name, String salutation, String add, String details,
MailingSystem mSys) {
        this.name = name;
        this.address = add;
        this.salutation = salutation;
        this.otherDetails = details;
        this.mailingSystem = mSys;
    }
    String getName() {
        return name;
    }
    MailingSystem getMailingSystem() {
        return mailingSystem;
    }
}
```

Refactoring Technique: Collapse Hierarchy



Refactoring Technique: Inline Class



Refactoring Technique: Remove Parameter



Code Smell: Refused Bequest

Refused Bequest occurs when a subclass inherits methods or data from a parent class that it does not need or want. The subclass is forced to inherit unnecessary behavior.

Refactoring Techniques

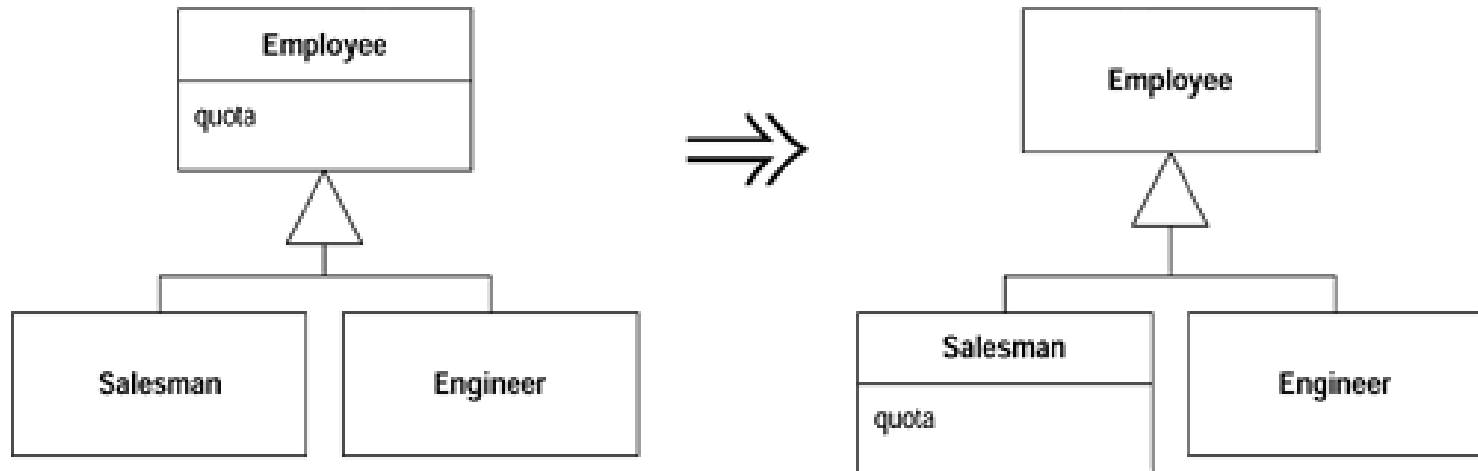
- Push Down Field/Method
- Replace Inheritance with Delegation



Code Smell: Refused Bequest (Cont'd)

```
public abstract class AbstractCollection...  
    public abstract void add(Object element);  
  
public class Map extends AbstractCollection...  
    // Do nothing because user must input key and value  
    public void add(Object element) {  
    }
```

Refactoring Technique: Push Down Method



Duplicate Code

Duplicate Code is the most common code smell in software development. It occurs when the same or similar code appears in multiple places. There are two types of duplication:

- **Obvious duplication:** Copy and paste of the same code.
- **Subtle duplication:** Similar algorithms or parallel inheritance hierarchies.

Refactoring Techniques:

- Extract Method
- Pull Up Field
- Form Template Method
- Substitute Algorithm



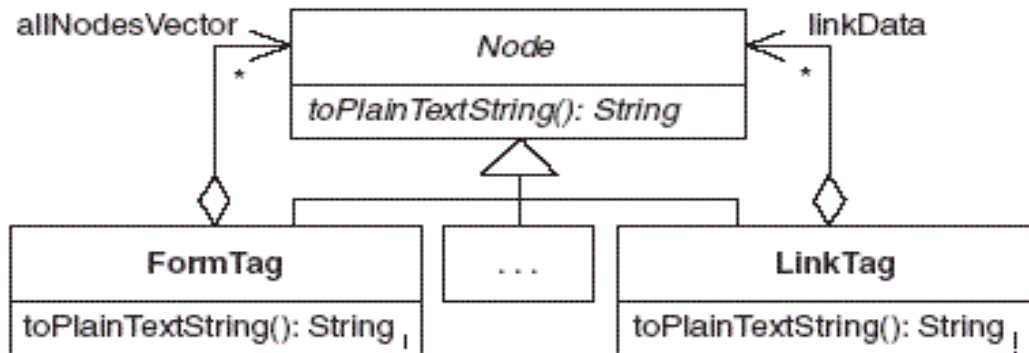
Duplicate Code (Cont'd)

Ctrl+C Ctrl+V Pattern

```
public static MailTemplate getStaticTemplate(Languages language) {
    MailTemplate mailTemplate = null;
    if(language.equals(Languages.English)) {
        mailTemplate = new EnglishLanguageTemplate();
    } else if(language.equals(Languages.French)) {
        mailTemplate = new FrenchLanguageTemplate();
    } else if(language.equals(Languages.Chinese)) {
        mailTemplate = new ChineseLanguageTemplate();
    } else {
        throw new IllegalArgumentException("Invalid language type specified");
    }
    return mailTemplate;
}

public static MailTemplate getDynamicTemplate(Languages language, String content) {
    MailTemplate mailTemplate = null;
    if(language.equals(Languages.English)) {
        mailTemplate = new EnglishLanguageTemplate(content);
    } else if(language.equals(Languages.French)) {
        mailTemplate = new FrenchLanguageTemplate(content);
    } else if(language.equals(Languages.Chinese)) {
        mailTemplate = new ChineseLanguageTemplate(content);
    } else {
        throw new IllegalArgumentException("Invalid language type specified");
    }
    return mailTemplate;
}
```

Duplicate Code (Cont'd)



```
StringBuffer textContents = new StringBuffer();
Enumeration e = allNodesVector.elements();
while (e.hasMoreElements()) {
    Node node = (Node)e.nextElement();
    textContents.append(node.toPlainTextString());
}
return textContents.toString();
```

```
StringBuffer sb = new StringBuffer();
Enumeration e = linkData.elements();
while (e.hasMoreElements()) {
    Node node = (Node)e.nextElement();
    sb.append(node.toPlainTextString());
}
return sb.toString();
```

Duplicate Code (Cont'd)

```
public int addCustomer( int userId, Customer  
newCustomer) {
```

```
    List<Customer> customerList = customers.get(userId);
```

```
    int customerId = (int) Math.random() * 1000;
```

```
    // TODO: Logic to find/generate customer id.
```

```
    newCustomer.setId(customerId);
```

```
    if (customerList == null) {
```

```
        customerList = new LinkedList<Customer>();
```

```
        customers.put(userId, customerList);
```

```
    }
```

```
    customerList.add(newCustomer);
```

```
    return customerId;
```

```
}
```

```
public int addTemplate( int userId, Template  
newTemplate) {
```

```
    List<Template> templateList = templates.get(userId);
```

```
    int templateId = (int) Math.random() * 1000;
```

```
    // TODO: Logic to find/generate template id.
```

```
    newTemplate.setId(templateId);
```

```
    if (templateList == null) {
```

```
        templateList = new LinkedList<Template>();
```

```
        templates.put(userId, templateList);
```

```
    }
```

```
    templateList.add(newTemplate);
```

```
    return templateId;
```

```
}
```

Duplicate Code (Cont'd)

Levels of duplication

- Literal
- Semantic
- Data Duplication
- Conceptual

Duplicate Code (Cont'd)

Literal Duplication

Exact copies of code are repeated in multiple locations

Example: Copy-pasting the same block of code without modifications across files or classes

Duplicate Code (Cont'd)

Code segments perform the same task or have the same logic but are written differently, often with slight variations in syntax or implementation.

Semantic Duplication

Example: Two methods that calculate a discount, but one uses a for loop while the other uses a while loop.

Duplicate Code (Cont'd)

Data Duplication

Duplication of data structures, constants, or configuration data across the codebase. It can cause inconsistency because the same data may need to be updated in multiple places.

Example: Hardcoding the same value (like tax rates or file paths) in multiple places.



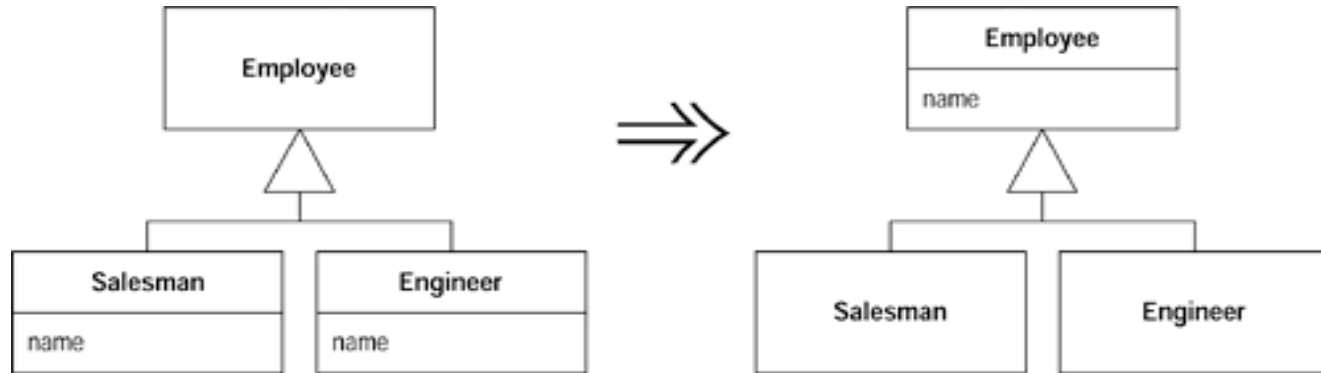
Duplicate Code (Cont'd)

Conceptual Duplication

Conceptual Duplication occurs when the same idea, concept, or responsibility is implemented multiple times in different places at a high-level design or architectural level.

Example: Multiple modules independently implementing their own logging mechanisms instead of using a shared logging service.

Refactoring Technique: Pull Up Field



Refactoring Technique: Substitute Algorithm

```
String foundPerson(String[] people){  
    for (int i = 0; i < people.length; i++) {  
        if (people[i].equals ("Don")){  
            return "Don";  
        }  
        if (people[i].equals ("John")){  
            return "John";  
        }  
        if (people[i].equals ("Kent")){  
            return "Kent";  
        }  
    }  
    return "";  
}
```

```
String foundPerson(String[] people){  
    List candidates = Arrays.asList(new String[]  
    {"Don", "John", "Kent"});  
    for (String person : people)  
        if (candidates.contains(person))  
            return person;  
    return "";  
}
```

Existing algorithm or piece of code can be replaced with a better or simpler one that achieves the same outcome.

This technique involves replacing the current algorithm with a new one that is clearer, more efficient, or easier to understand and maintain.

Code Smell: Switch Smell

Switch Smell is a code smell that occurs when repeated switch statements or multiple if-elif-else conditions are used throughout the code to handle different cases. It increases code complexity, makes maintenance difficult, and indicates that polymorphism is not being used effectively.

Refactoring Techniques

- Replace Type Code with Polymorphism
- Replace Type Code with State / Strategy
- Replace Parameter with Explicit Methods
- Introduce Null Object.



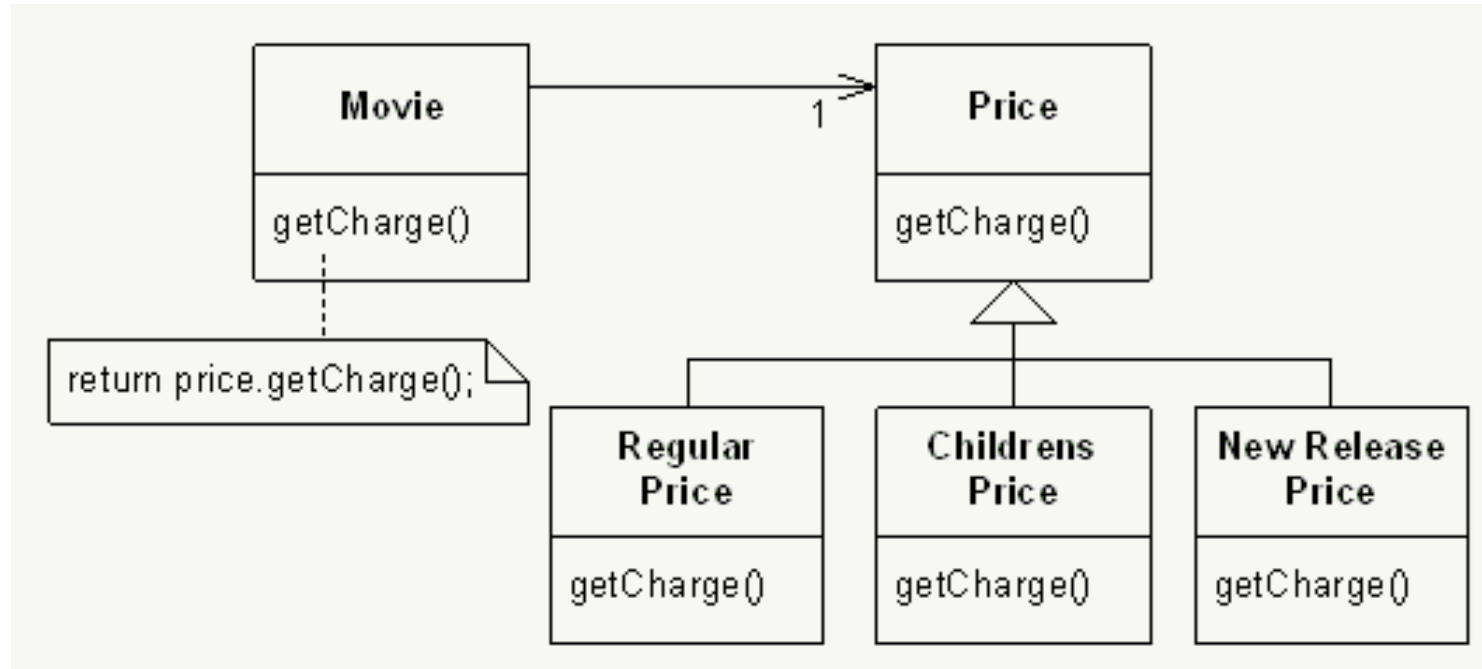
Code Smell: Switch Smell (Cont'd)

```
if( "=".equalsIgnoreCase(operator.trim())){  
    for(int i=0;i<ruleCriteria.getCriteriaSize();i++){  
        if(ruleCriteria.getCriteria(i).equalsIgnoreCase("Name"))  
            criteriaCompareStrategy[i] = new Integer(nameFlag);  
        else if(ruleCriteria.getCriteria(i).equalsIgnoreCase("City"))  
            criteriaCompareStrategy[i]=new Integer(cityFlag);  
        else if(ruleCriteria.getCriteria(i).equalsIgnoreCase("Address"))  
            criteriaCompareStrategy[i]=new Integer(addressFlag);  
        else if(ruleCriteria.getCriteria(i).equalsIgnoreCase("Age"))  
            criteriaCompareStrategy[i]=new Integer(ageFlag);  
        else if(ruleCriteria.getCriteria(i).equalsIgnoreCase("Income"))  
            criteriaCompareStrategy[i]=new Integer(incomeFlag);  
        else if(ruleCriteria.getCriteria(i).equalsIgnoreCase("TotalPurchase"))  
            criteriaCompareStrategy[i]=new Integer(spendingFlag);  
    }  
}
```

Code Smell: Switch Smell (Cont'd)

```
while (rentals.hasMoreElements()) {  
    double thisAmount = 0;  
    Rental each = (Rental)rentals.nextElement();  
  
    //determine amounts for each line  
    switch (each.getMovie().getPriceCode()) {  
        case Movie.REGULAR:  
            thisAmount += 2;  
            if (each.getDaysRented() > 2)  
                thisAmount += (each.getDaysRented() - 2) * 1.5;  
            break;  
        case Movie.NEW_RELEASE:  
            thisAmount += each.getDaysRented() * 3;  
            break;  
        case Movie.CHILDRENS:  
            thisAmount += 1.5;  
            if (each.getDaysRented() > 3)  
                thisAmount += (each.getDaysRented() - 3) * 1.5;  
            break;  
    }  
}
```

Refactoring: Replace Type Code with Polymorphism



Instead of using many conditions, create separate classes and let each class handle its own behavior using **polymorphism**.

Refactoring: Replace Parameter with Method

```
void setValue (String name, int value) {  
    if (name.equals("height"))  
        this.height = value;  
    if (name.equals("width"))  
        this.width = value;  
}
```

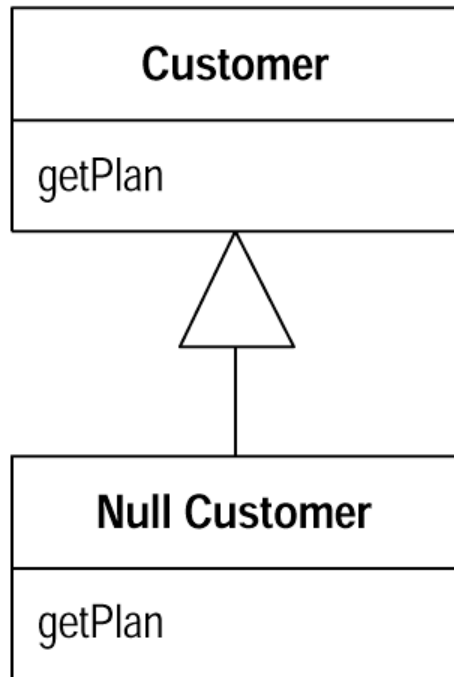
```
void setHeight(int h) {  
    this.height = h;  
}  
void setWidth (int w) {  
    this.width = w;  
}
```

When a method receives a parameter only to decide what value to use, replace that parameter with separate methods that directly describe the action.

Refactoring: Introduce Null Object

```
Customer customer = site.getCustomer();  
BillingPlan plan;  
if (customer == null)  
    plan = BillingPlan.basic();  
else  
    plan = customer.getPlan();
```

```
Customer customer = site.getCustomer();  
BillingPlan plan = customer.getPlan();  
  
public BillingPlan getPlan(){  
    return BillingPlan.basic();  
}
```



Refactoring: Large Class

- A Large Class is a class that has too many responsibilities. Like people, classes suffer when they take on too many responsibilities, making the code difficult to understand, maintain, and modify.
- A large class is often called a God Object because it tries to handle many unrelated tasks. Fowler and Beck note that the presence of too many instance variables usually indicates that a class is trying to do too much. In general, large classes typically contain too many responsibilities.

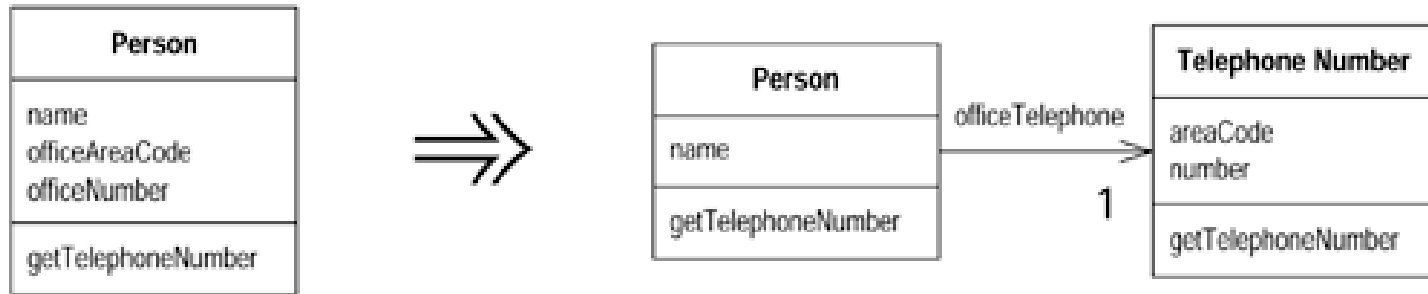
Refactoring techniques:

- Extract Class
- Replace Type Code with Class/Subclass
- Replace Type Code with State/Strategy
- Replace Conditional with Polymorphism
- Extract Interface



Refactoring: Extract Class

Extract Class is used when one class has too many responsibilities. Related fields and methods are moved into a new class, making each class responsible for only one task.



The Person class stores:(Before Extract)

- Person information (name)

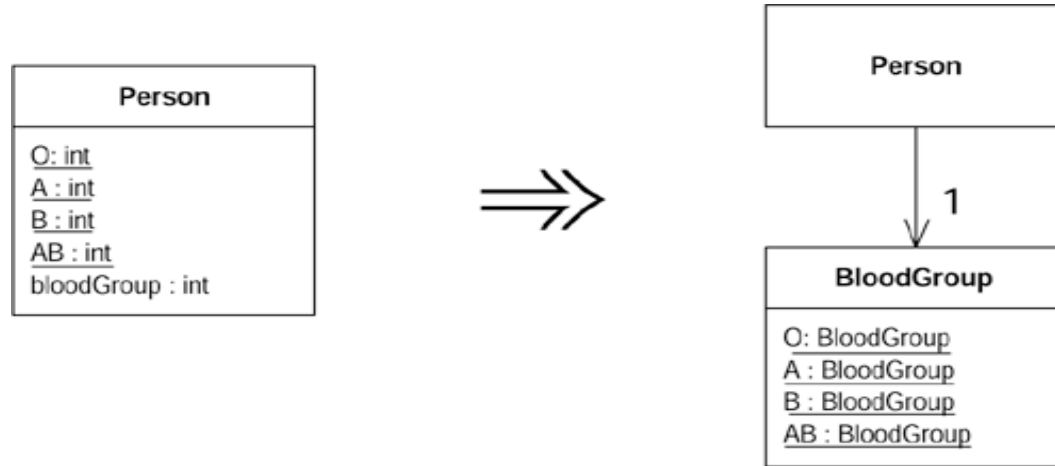
- Telephone information (officeAreaCode, officeNumber)

The Person class has **two responsibilities**:

- Managing person details.

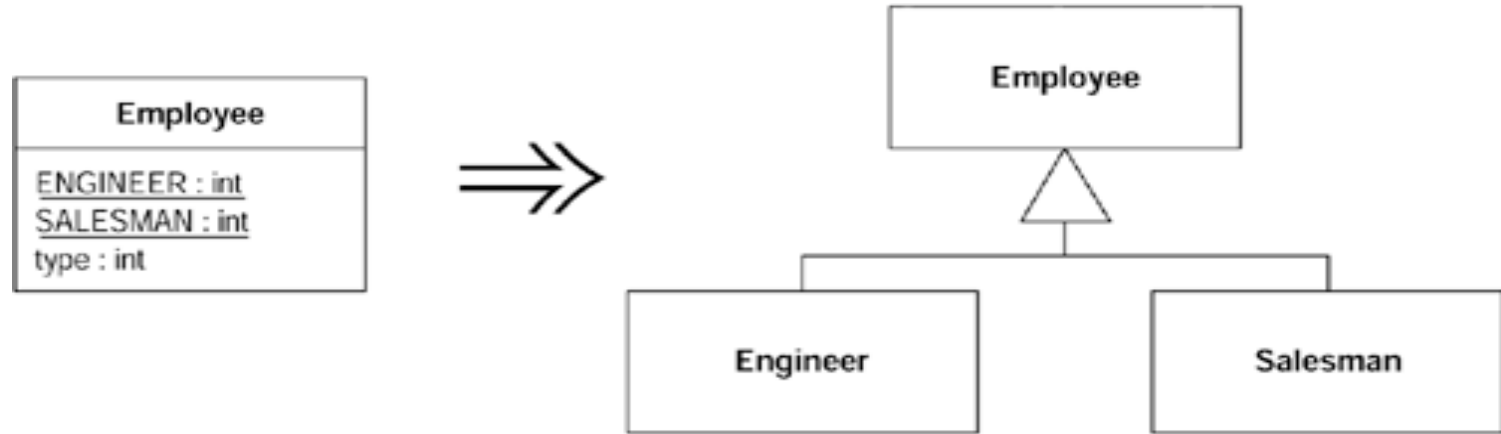
- Managing telephone details.

Refactoring: Replace Type Code with Class



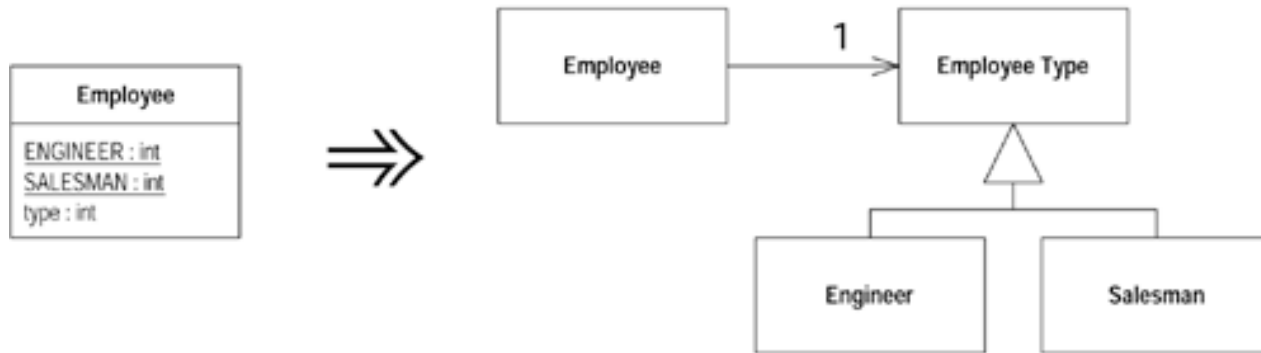
Replace Type Code with Class replaces a primitive type (such as int or String) with a dedicated class that represents the type. It improves readability, type safety, and maintainability by giving meaningful names instead of magic values.

Refactoring: Replace Type Code with Subclasses

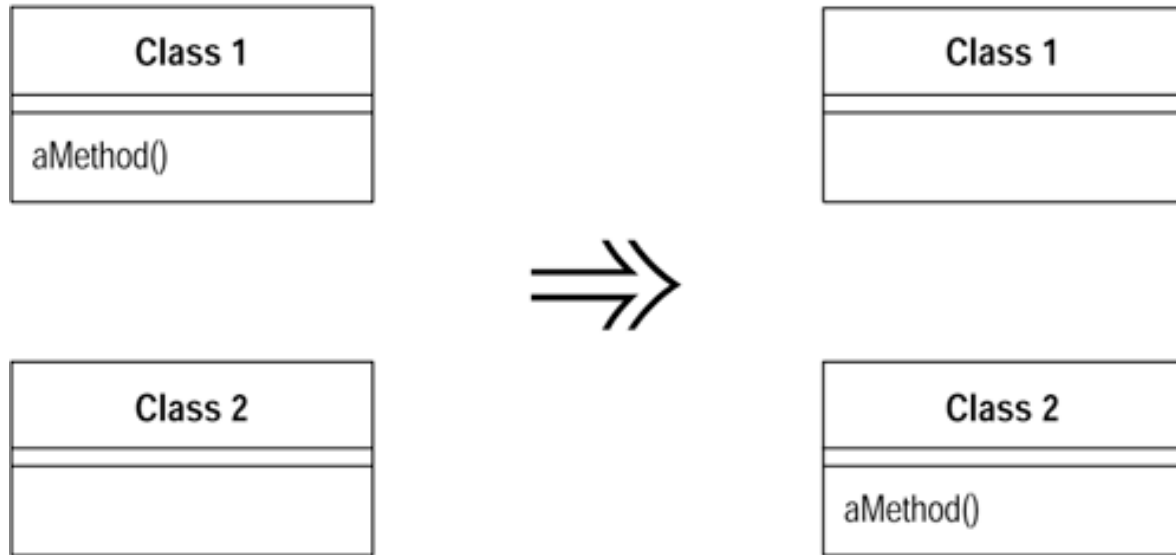


Instead of storing a type code (such as ENGINEER or SALESMAN) inside one class, create a subclass for each type.

Refactoring: Replace Type Code with State/Strategy



Refactoring: Move Method/Field



Data clumps

- Groups of data that frequently appear together in multiple parts of a program

Remedy: Extract class

```
def create_customer(name, street, city, zip_code):  
    print(name, street, city, zip_code)  
  
def send_invoice(name, street, city, zip_code):  
    print("Invoice sent to", street, city, zip_code)
```

```
class Address:  
    def __init__(self, street, city, zip_code):  
        self.street = street  
        self.city = city  
        self.zip_code = zip_code  
  
def create_customer(name, address):  
    print(name, address.city)  
  
def send_invoice(name, address):  
    print("Invoice sent to", address.street)
```

Data clumps (Cont'd)

```
public class AppointmentScheduler {  
    public void ScheduleAppointment(string firstName,  
        string lastName, string dateOfBirth, string address,  
        DateTime appointmentDate) {  
        // Code to schedule an appointment  
    }  
}
```

```
public class PatientRecords {  
    public void CreateRecord(string firstName,  
        string lastName, string dateOfBirth, string  
        address) {  
        // Code to create a new patient record  
    }  
}
```

```
public class BillingSystem {  
    public void ProcessPayment(string firstName, string lastName, string dateOfBirth, string  
        address, decimal amount) {  
        // Code to process payment  
    }  
}
```


Contrived complexity

Contrived Complexity occurs when developers force the use of overly complicated design patterns where a simpler design would suffice, making the code unnecessarily difficult to understand and maintain.

"Any intelligent fool can make things bigger, more complex, and more violent. It takes a touch of genius...and a lot of courage to move in the opposite direction." ~ E.F. Schumacher



How to start identifying code smells and refactoring?

Refactoring is tough in old code and most of the developers failed as they started it in wrong way..

- Before starting refactoring, start writing Unit Testing code.
- Apply refactoring in new code instead of old ones.
- Start with the following three code smells, refactor them without approval from anyone:
 - ❖ Inappropriate naming
 - ❖ Duplicate code
 - ❖ Long method

Thank you

